

Exploring Data Acquisition and Decision Analysis Through Information Entropy by using Battleship

[BEng] Mechanical Engineering 2025

Xiaoyu Zhang

StudentID: 11095717

Email: Xiaoyu.zhang-19@student.manchester.ac.uk

Project Supervisor: Dr. Peter Hollingsworth

The University of Manchester

Individual 3rd Year Project

Department of Mechanical, Aerospace Engineering

Project Groups: Autonomous Systems, Aerodynamics and Space (AAS)

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Peter Hollingsworth, for his guidance, insightful discussions, and valuable ideas throughout this project. His support in refining the algorithm and analyzing the entropy-based decision-making process has been instrumental in shaping the research.

Content

1. Introduction	1
1.1 Battleship	1
1.2 Background and Motivation	2
1.3 Research Significance and Scope	3
1.4 Aims and Objectives	3
2. Literature Review	5
2.1 Theoretical Foundations in Information Theory	5
2.2 Algorithmic Strategies and Optimality	5
2.3 Entropy-Driven Decision-Making in the Current Study	5
2.4 Synthesis and Gaps	6
3. Method	7
3.1 Simulation Setup	7
3.2 Algorithm Design	9
3.3 Entropy Modeling	17
3.4 Wavelet Transform for Entropy Analysis	18
4. Result	20
4.1 Random Strategy: Result and Analysis	20
4.2 Local Possibility Strategy: Result and Analysis	27
4.3 Advance Global Entropy Analyze	35
5. Discussion & Conclusion	41
5.1 Further Work	43
5. Management	44
6.1 Original Management	44
6.2 Reflection and Planning Analysis	46
6.3 Risk Assessment and Future Work Recommendations	46
6. Reference	48
Appendix	49

List of figures

Figure 1 Battleship Physical Gameboard	2
Figure 2 Randomly generate a gameboard (0 is empty, 1 is occupied).....	7
Figure 3 Flowchart for Local Possibility Estimation.....	13
Figure 4 Turn distribution of 10,000 random strategy games	20
Figure 5 Global entropy trend over a full random game, victory at Turn 186	22
Figure 6 Global entropy trend over a full random game, Victory at Turn 193.....	22
Figure 7 Random CWT of entropy signal from Figure 6	24
Figure 8 Random CWT of entropy signal from Figure 7	24
Figure 9 Normal Distribution of Turns for Local Possibility Strategy.....	28
Figure 10 Global Entropy Curve for 43 Turns	31
Figure 11 Global Entropy Curve for 97 Turns	31
Figure 12 Global Entropy Curve for 145 Turns	31
Figure 13 Global Entropy CWT Chart of 43 Winning Turns.....	32
Figure 14 Global Entropy CWT Chart of 97 Winning Turns.....	32
Figure 15 Global Entropy CWT Chart of 145 Winning Turns.....	32
Figure 16 Local Entropy Curve, 43 Turns	33
Figure 17 Local Entropy Curve, 97 Turns	33
Figure 18 Local Entropy Curve, 145 Turns	33
Figure 19 Local Entropy CWT, 43 Turns.....	34
Figure 20 Local Entropy CWT, 97 Turns.....	34
Figure 21 Local Entropy CWT, 145 Turns.....	34
Figure 22 Advanced global entropy heatmap, Associated with Figure 23 Data of Each Ship. .	36
Figure 24 Entropy Heatmap at Turn 1, Turn 18, Turn 34 and Turn 41	38

Lists of Table

Table 4.2.2a	38
Table 4.2.2b	39
Table 4.2.2c	40
Table 4.2.2d	41

Abstract

Battleship, a classic grid-based game of hidden target discovery, offers a well-contained yet strategically rich environment for testing information-driven decision-making. Its simple rules, spatial structure, and clear win conditions make it an ideal model for studying uncertainty reduction and search efficiency—concepts that are central to engineering disciplines such as robotics, diagnostics, and autonomous exploration.

This project applies information theory to optimize targeting strategies in a simulated 14×14 Battleship grid populated with five ships of varying sizes and orientations. Three algorithmic approaches are implemented and compared: a purely random search method, a local probability strategy that reacts to nearby hits, and an advanced global entropy method based on full-board permutation analysis. Local entropy refers to the uncertainty reduction based on immediate surroundings (e.g., near previous hits), whereas global entropy evaluates the overall information state across the board using statistical permutations.

Entropy values are calculated and updated after each turn, with heatmaps and wavelet visualizations used to track and interpret evolving patterns. These tools reveal how each strategy manipulates the entropy landscape over time and highlight the superior performance of the global algorithm in minimizing uncertainty and reducing the number of moves to victory.

The project demonstrates how combining global and local probabilistic reasoning can enhance search-based decision systems. The developed framework—including automated simulation, entropy quantification, and dynamic visualization—can be extended to real-world engineering applications where efficient information acquisition and action selection are critical.

1. Introduction

1.1 Battleship

Battleship is a classic turn-based strategy game in which players attempt to locate and destroy an opponent's hidden fleet on a discrete grid. Figure 1 shows the general shape of the game board. Each fleet consists of multiple ships placed horizontally or vertically, with the player making sequential guesses to uncover their positions. A hit provides partial confirmation of a ship's location, while a miss yields negative information.

In this project, the game is implemented on a 14×14 grid with five ships of varying lengths. Ships are randomly placed without overlap, and the player progresses by selecting coordinates to strike. Feedback from each move updates the player's knowledge of the game state.

This simple yet information-rich structure makes Battleship an excellent model for studying decision-making under uncertainty. The need to infer hidden states from partial observations parallels real-world problems such as diagnostic testing, path planning, or tactical search operations. Each action contributes to the accumulation of information, affecting subsequent decisions and outcomes.

By simulating the game using different strategies—ranging from fully random guesses to probability-informed targeting—this project treats Battleship as a platform for analyzing the dynamics of information acquisition. Specifically, it investigates how local and global entropy evolve in response to probabilistic decision-making, providing a foundation for entropy-based modeling in broader engineering contexts.

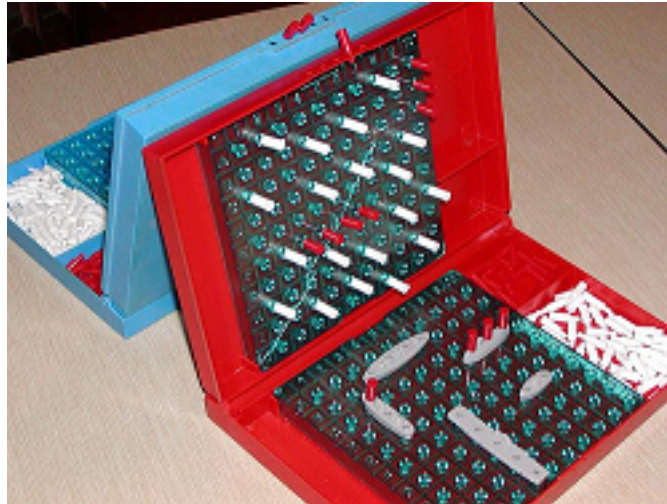


Figure 1 Battleship Physical Gameboard

1.2 Background and Motivation

In modern engineering systems—particularly in robotics, automated manufacturing, and inspection processes—decision-making under uncertainty plays a central role. Mechanical systems are increasingly integrated with intelligent algorithms that must operate efficiently with incomplete or evolving information. Whether it is a robotic arm locating a defective component, a CNC system optimizing tool paths in real time, or an autonomous vehicle detecting obstacles, these systems must balance exploration and precision based on limited feedback.

The classic strategy game Battleship provides a simplified yet insightful framework for modeling such scenarios. Players are tasked with identifying hidden targets on a grid using sequential guesses and feedback, mimicking real-world engineering problems where information must be inferred from noisy or sparse data. The turn-based structure of the game parallels iterative control and sensing mechanisms in mechanical systems.

Information theory—particularly the use of Shannon entropy—offers a mathematical foundation for quantifying uncertainty in these systems. By treating each strike in the game as an information-gathering action, this project explores how entropy evolves under different strategies. More importantly, it evaluates how entropy-informed decision-making can improve efficiency, reduce unnecessary exploration, and maximize target identification, which are all critical concerns in engineering optimization.

Through the analysis of local and global probability-based strike algorithms, the project aims to draw parallels between strategic gameplay and engineering decision systems. In doing so, it provides a testable platform to simulate how entropy-guided reasoning could benefit real-world mechanical applications, from inspection automation to intelligent sensor deployment and fault diagnosis.

1.3 Research Significance and Scope

This project holds significance in both theoretical and applied contexts. On the theoretical side, it presents a structured environment in which the concept of entropy can be directly visualized and measured across iterative decision processes. It bridges the abstract concept of information gain with concrete gameplay events such as target acquisition and strike accuracy. These relationships are rarely quantified so directly in more complex, real-world systems.

On the applied side, the project provides insight into how probabilistic reasoning—particularly the use of both local and global probability distributions—can enhance the performance of search algorithms. This has strong relevance to mechanical and automation engineering, where systems must often make spatial decisions based on incomplete sensor data. Examples include automated inspection in manufacturing, robotic scanning for material defects, or pattern exploration in multi-axis platforms. By simulating entropy evolution on a bounded grid, the project offers a conceptual tool that can be extended to path optimization, real-time target localization, or efficient resource deployment.

The scope of the study is limited to a single-player variant of Battleship on a 14×14 grid with five ships. The opponent's ship locations are randomly generated, and the player algorithm operates without prior knowledge. Once a ship is fully hit, it is publicly announced as "sunk", simulating the feedback loop in many real-world applications. Strategies analyzed include fully random attacks, entropy-guided attacks based on local hit analysis, and an advanced model that integrates global permutation-based probability. Visualization tools such as entropy heatmaps and per-turn entropy tracking are implemented to support deeper interpretability of algorithm performance.

1.4 Aims and Objectives

1.4.1 Aims

To investigate how entropy-based decision-making strategies influence the efficiency of gameplay in the Battleship board game, and to explore the role of global and local probability distributions in guiding optimal search under uncertainty.

1.4.2 Objectives

To achieve this aim, the project is structured around the following testable objectives:

1. **Identify and compare multiple strike algorithms**, including:
 - Random (uninformed) targeting,
 - Local probability-based entropy guidance,
 - Global permutation-based entropy calculation.
2. **Track and visualize the entropy dynamics associated with each algorithmic strategy throughout the game, using per-turn entropy analysis and heatmap-based representations.**
 - 2.1 Conduct a comparative evaluation of the algorithms by analyzing entropy progression, prediction accuracy, and game performance metrics such as turns required to achieve victory.
 - 2.2 Examine the evolution of local and global entropy under each approach, identifying recurring patterns and quantifying their influence on decision quality and strategic effectiveness.
3. **Explore the engineering implications** of entropy-driven strategies in real-world applications such as inspection automation, robotic exploration, or fault localization.

2. Literature Review

The study of Battleship as a decision-making framework has been approached through diverse methodologies, ranging from information theory to algorithmic optimization. This section synthesizes key contributions from existing literature and contextualizes the current research within this landscape.

2.1 Theoretical Foundations in Information Theory

McCue’s information-theoretic analysis (McCue, 2005) establishes *Battleship* as a paradigm for quantifying uncertainty reduction. By framing ship placement as a problem of encoding configurations (≈ 35 bits for a 10×10 grid), McCue demonstrates how Shannon entropy measures the efficiency of search strategies. His work bridges game mechanics to real-world applications, such as the search for the USS *Scorpion*, where Bayesian priors and Monte Carlo simulations optimize probabilistic targeting. Notably, McCue (2005) highlights the non-intuitive relationship between information entropy and spatial dependencies, emphasizing that localized probability updates (e.g., post-hit targeting) accelerate uncertainty reduction—a principle central to the entropy-driven algorithms in the current study.

2.2 Algorithmic Strategies and Optimality

Audinot et al. (2014) focus on deterministic strategies against a random opponent, computing the minimal average number of shots (ϕ) required to sink ships. Their recursive algorithm constructs decision trees to minimize total search steps, revealing that greedy strategies (e.g., targeting high-probability cells) are suboptimal in certain configurations. For example, on a 3×4 grid with a ship of length 2, the optimal strategy achieves an average of $53/12$ shots, outperforming greedy approaches ($54/12$). While their work provides exact ϕ values for small grids (e.g., 11.3 shots for a 4×5 grid with ships of lengths 2, 3, 4), computational limitations restrict analysis of the standard 10×10 game to bounds ($30.8 < \phi < 55.6$). This underscores the challenge of scaling entropy-based methods to larger grids—a gap partially addressed in the current research through dynamic entropy visualization.

2.3 Entropy-Driven Decision-Making in the Current Study

The present research diverges by integrating real-time entropy analysis into *Battleship* gameplay. While prior studies model entropy statically (e.g., McCue, 2005) or algorithmically (e.g., Audinot et al., 2014), this work employs Continuous Wavelet Transform (CWT) to analyze entropy fluctuations across game rounds. By contrasting global entropy (overall uncertainty) and local entropy (post-hit targeting), the study reveals how adaptive strategies reduce search inefficiencies. For instance, the probability-based algorithm reduces average turns from ~ 194 (random) to ~ 55 by prioritizing high-entropy regions post-hit, aligning with McCue’s (2005) emphasis on Bayesian updates. However, the current approach uniquely visualizes entropy dynamics (e.g., CWT heatmaps in Figures 4–5), offering insights into temporal-frequency patterns that traditional methods overlook.

2.4 Synthesis and Gaps

While Audinot et al.’s (2014) work provides a theoretical benchmark for optimality, its computational complexity limits practical applicability. Conversely, McCue’s (2005) information-theoretic model lacks granularity in real-time decision adjustments. The current study bridges these gaps by operationalizing entropy as a dynamic metric, enabling iterative strategy refinement. However, limitations persist:

1. **Rule Variations:** Neither the current study nor prior works fully explore rule modifications (e.g., announcing sunk ships), which could alter entropy propagation.
2. **Multi-Target Complexity:** Audinot et al.’s (2014) counterexamples highlight challenges in multi-ship scenarios, while the current research identifies entropy “spikes” around clustered targets but lacks robust solutions.
3. **Scalability:** All studies struggle with scaling to larger grids, though the use of Python simulations here offers a flexible framework for future extensions.

In summary, this research advances *Battleship* strategy analysis by merging theoretical entropy principles with dynamic computational tools, offering a novel perspective on uncertainty management in constrained search environments.

3. Method

3.1 Simulation Setup

To facilitate entropy-based analysis and algorithm evaluation, a controlled digital simulation of the Battleship game was developed in Python. The game environment follows a single-player model, where the algorithm plays as the attacker and the opponent's ship layout is randomly generated at the beginning of each session. The core structure and parameters of the simulation are described as follows:

Grid Dimensions: The game board is implemented as a 15×15 NumPy matrix [0...14][0...14]. However, only the internal region from [0...13][1...14] is used as the **valid playable area**, representing a 14×14 battlefield. The first column [x=0] and the last row [y=14] serve as **coordinate markers** for visualization purposes and are excluded from ship placement and strike calculations. This design replicates a conventional Cartesian coordinate system, with (x=1, y=0) as the bottom-left playable cell and (x=14, y=13) as the top-right. (Figure 2)

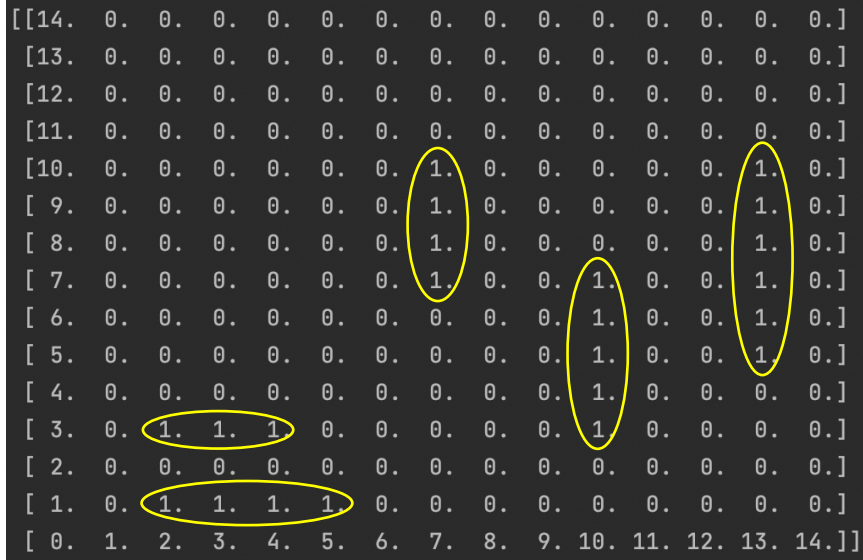


Figure 2 Randomly generate a gameboard (0 is empty, 1 is occupied)

Ship Configuration: Five ships (Data shows in Table 1) of varying lengths are randomly placed on the board without overlapping. Each ship can be oriented either horizontally or vertically. The system records each ship's type, orientation, and occupied coordinates, which are dynamically updated during gameplay.

Table 1 Data of Each Ship

Ship Type	Number of Grids	Number of Ship	Shape
Cruiser	3	1	
Destroyer	4	1	
Submarine	4	1	
Battleship	5	1	
Carrier	6	1	

Attack Protocol: On each turn, the algorithm selects a coordinate (x, y) within the valid area to strike. The cell is updated according to the following logic:

- If it contains a ship part, it is marked as a **hit**.
- If it is empty, it is marked as a **miss**.
- If it has been targeted before, the algorithm repeats the selection process until a valid coordinate is found.

Game Termination: The game continues until all five ships are completely destroyed. A ship is declared **sunk** when all its associated coordinates have been hit. Upon sinking, the system announces the ship type and direction. This explicit feedback mechanism allows entropy values to be recalculated more precisely.

Entropy Tracking and Visualization: During every turn, entropy values—whether local or global—are calculated and stored on a 2D grid. These values are then visualized using heatmaps, enabling an intuitive interpretation of information flow and algorithmic behavior.

Repeatability and Batch Simulation: The simulation framework is designed for repeatable batch runs. Each algorithm is tested across multiple randomized ship layouts to assess statistical consistency and eliminate bias from isolated cases.

This simulation setup ensures a stable foundation for evaluating various entropy-guided strategies under identical structural constraints. It bridges abstract probabilistic theory with measurable gameplay outcomes, allowing deep insight into how information is accumulated and exploited during sequential search.

3.2 Algorithm Design

3.2.1 Random Strike Strategy

The random strike strategy represents the simplest form of decision-making in the Battleship simulation. At each turn, the algorithm selects a coordinate at random from the remaining untargeted cells within the valid playable area $[0 \dots 13][1 \dots 14]$. If the selected coordinate has already been attacked, the algorithm continues to sample randomly until a valid strike location is found.

This method is entirely non-inferential—it does not incorporate any prior strike results, spatial probabilities, or history of hits. Every available cell is considered equally likely to contain part of a ship. As such, it provides a neutral baseline for evaluating the performance of more sophisticated algorithms.

Within the scope of this project, the random strategy is used to:

Establish a benchmark for measuring efficiency and outcome across different algorithms.

Simulate a scenario in which no information about the board is used.

Provide a reference point for comparing the structure of strike distributions and outcome patterns.

This strategy does not make use of hit feedback to inform future decisions and assumes no inherent structure or pattern in the ship placement.

3.2.2 Local Possibility Strategy

The local possibility strategy introduces an information-guided mechanism that responds dynamically to feedback from previous strikes. In contrast to the random method, this approach incorporates partial knowledge—specifically, successful hits—to guide follow-up actions and concentrate strikes in likely ship-adjacent regions.

This algorithm operates in two distinct phases:

Search Phase (Randomized):

Before any successful hit is made, the algorithm selects target coordinates at random from the pool of untargeted cells. This phase is identical to the random strategy, representing an uninformed exploration of the board.

Targeting Phase (Possibility-Based):

Once a hit is confirmed, the algorithm transitions into a targeting mode, focusing on the immediate vicinity of the hit coordinate. It evaluates adjacent cells along the row and column directions to estimate the most likely continuation of the ship. Unattacked cells that align linearly with the known hit are prioritized. If a second hit is found, the direction is locked in, and the search continues in that direction until the ship is sunk.

After a ship is sunk, the algorithm resets to the randomized search phase and repeats the process for locating the next ship.

This method does not rely on global probability models or entropy maps but instead uses local spatial reasoning and feedback-driven heuristics to refine its decisions. It reflects intuitive human strategies in search tasks, offering improved efficiency over pure randomness while remaining computationally lightweight.

Mathematical Model for Local Possibility Estimation

The local possibility model implemented in this project is derived from combinatorial evaluation of all feasible ship placements in the vicinity of confirmed hit coordinates. It assumes that ships are placed in contiguous linear arrangements either horizontally or vertically.

Let l be the length of a candidate ship segment and let $H = \{(x_1, y_1), (x_2, y_2), \dots\}$ be the set of currently confirmed hit coordinates.

1. Permutation Calculation (Algorithm 1)

We define the total number of valid ship placements (permutations) given the hit set H as:

$$P = P_H + P_V \quad (1)$$

where:

P_H : Number of valid horizontal permutations aligned with H

P_V : Number of valid vertical permutations aligned with H

These are calculated by scanning available spaces in both directions from the hit cluster and verifying space for full ship length l , subject to boundary and conflict conditions.

The Permutation Calculator algorithm determines the number of valid placements for survival ships after a hit is registered on the gameboard. The flow begins with iterating through all ship types (No.1 to No.5) and skipping those already sunk. For each remaining ship, the algorithm checks whether there is sufficient empty space around the hit coordinate in both the horizontal and vertical directions. This "cross area" is analyzed for uninterrupted zero-value cells, representing valid regions for potential ship placement.

If the surrounding space is insufficient for a ship of the current length, the algorithm skips it. Otherwise, it proceeds to calculate all valid permutations, counting how many ways the ship can be placed within the available space. These permutation values are recorded separately for both horizontal and vertical directions.

The results are stored in a list representing the permutation distribution of surviving ships, which serves as the basis for calculating global entropy. This step enables the model to quantify spatial uncertainty dynamically and prioritize regions with high probabilistic relevance for future strikes.

This visual logic (Figure 4, permutation calculator part) demonstrates how the algorithm combines spatial validation, directional checking, and iterative updating to generate a real-time probability model—a key mechanism in the global entropy-based strategy.

2. Weight Accumulation (Algorithm 2)

For each valid permutation, we increment the score of the cells it occupies:

$$W_{(x,y)} += 1 \quad \forall (x,y) \in \textit{cells covered by a valid permutation} \quad (2)$$

3. Local Probability Assignment

Once all permutations are processed, the weight matrix is normalized into a local probability map:

$$P_{(x,y)} = \frac{W_{(x,y)}}{P} \text{ if } P > 0 \quad (3)$$

This matrix $P_{(x,y)}$ serves as the localized likelihood estimate for selecting the next target coordinate. It is recalculated every time a new hit is confirmed and is used to prioritize the most promising directions.

Following the output of the Permutation Calculator (Figure 4, permutation calculator part), the second stage focuses on translating those permutation counts into local probability distributions across the board—providing the foundational data for entropy calculation.

The algorithm begins by retrieving each surviving ship's possible placement count in both horizontal and vertical directions. For each direction and ship, it identifies the farthest reachable coordinate from the latest hit location—either to the left/right (for horizontal) or top/bottom (for vertical). This farthest valid cell becomes the starting point for permutation-based estimation.

From this point, the algorithm traverses each possible ship placement by marking a running counter over valid coordinates—simulating all valid placements. Every time a ship could fit into a new legal position, each involved cell's "occurrence" is incremented, resulting in a cumulative numerator distribution matrix.

Once all permutations are accumulated, the local probability matrix is computed by dividing each cell's numerator value by the total number of valid permutations previously calculated (from Figure 4, weight calculator part). This yields a normalized 2D matrix of local strike probabilities.

Finally, this matrix becomes the input for Shannon Entropy Calculation, producing the local entropy heatmap for that game turn. The result is a dynamic 2D entropy map that reflects not only valid placements but also how the player's current knowledge (e.g., prior hits, blocked zones) influences strategic probability.

This dual-stage algorithm (Figures 4) merges geometric validation, iterative accumulation, and probabilistic normalization, forming the core of the local-entropy strike logic—an efficient and scalable method for uncertainty-driven decision-making.

3.2.3 Global Permutation Strategy (Algorithm 3)

The global permutation strategy represents the most comprehensive and information-intensive method developed in this project. Unlike the random and local possibility strategies, which either do not utilize feedback or only focus on nearby hits, this approach systematically evaluates the **entire game board** to identify the most probable locations for all remaining ships.

This method is built upon a grid-wide **enumeration of valid ship placements** for each survival ship type. At every turn, the algorithm scans the board and, for every empty cell, calculates how many valid horizontal and vertical ship permutations could occupy that cell. These permutation counts are accumulated in a probability matrix, which reflects the likelihood of each cell being part of any remaining ship.

The core steps are as follows:

1. Board-Wide Permutation Search

For each survival ship of length l , the algorithm iterates through all possible horizontal and vertical positions on the board. For each valid placement (i.e., not overlapping previous misses or sunken ships), the cells covered by the ship are incremented in a cumulative permutation matrix.

2. Cumulative Possibility Map

This cumulative matrix serves as a raw indicator of which cells are involved in the greatest number of valid placements. Higher values indicate higher possibility based on remaining ship sizes and board constraints.

3. Global Probability Normalization

After all ships have been considered, the matrix is normalized by the total number of permutations, yielding a **global strike probability map**:

$$P_{(x,y)} = \frac{\textit{Number of permutations involving } (x,y)}{\textit{Total number of permutations over board}} \quad (4)$$

3.2.4 Target Selection

The algorithm then selects the cell with the **maximum probability value** to attack.

This strategy leverages the maximum amount of available information: known hits, previous misses, sunk ships, and board boundaries. It produces a complete heatmap for each turn and dynamically adapts as the game progresses. While computationally more expensive than local heuristics, it offers a robust probability landscape that supports more globally optimized decision-making.

3.3 Entropy Modeling

In order to evaluate the efficiency and information gain associated with different strike strategies, this project incorporates an entropy-based analysis framework. Entropy, as defined in information theory by Shannon, is used to quantify the level of uncertainty in a probability distribution. Within the context of Battleship, entropy is used to measure the uncertainty of the board state at each turn—specifically, the degree to which remaining ship positions can be predicted based on available information.

The Shannon entropy H of a 2D probability distribution over the game board is defined as:

$$H = - \sum_{(x,y)} p(x,y) \log_2 p(x,y) \quad (5)$$

where:

$p(x,y)$ is the normalized strike probability at coordinate (x,y)

- The summation is taken over all valid cells in the board ($[0 \dots 13][1 \dots 14]$)
- Cells with zero probability are excluded to avoid undefined logarithmic terms

Entropy Applications in the Project

Entropy is computed at every turn using the probability matrix generated by the current strike algorithm. Depending on the strategy, the source of probabilities differs:

- For the random strategy, the distribution is uniform over untargeted cells.
- For the local possibility strategy, the distribution is built from feasible continuations of a partially hit ship.
- For the global permutation strategy, the distribution reflects all valid ship placements given the entire board state.

Probability Input per Strategy

Although the entropy calculation method remains constant, the way the probability matrix $p(x, y)$ is constructed differs by strategy:

Random Strategy:

Probabilities are uniformly distributed over all untargeted cells. No positional inference or history is used.

Local Possibility Strategy:

Probabilities are assigned only to a small subset of cells surrounding recent hits. The matrix remains sparse until further information is gathered.

Global Permutation Strategy:

Probabilities are derived from a full enumeration of all valid ship placements remaining on the board. Every untargeted cell may carry a nonzero value depending on its overlap with feasible permutations.

Entropy is computed at each turn using the respective $P(x, y)$ matrix from the active strategy. These entropy values are stored for further comparative analysis and visualization in subsequent sections.

3.4 Wavelet Transform for Entropy Analysis

3.4.1 Role of Wavelet Transform in Battleship Strategy

Wavelet Transform (WT) offers a time-frequency decomposition tool ideal for capturing localized entropy changes in dynamic systems like *Battleship*. Unlike the Fourier Transform, which only reveals global frequency content, WT identifies both transient and persistent features, such as sudden entropy spikes after hits or patterns caused by clustered ships (Mallat, 1999).

In this project, WT enables a multi-resolution view of the entropy distribution. High-frequency components correspond to sharp, local changes (e.g., concentrated hits), while low-frequency components reflect broader trends in ship placement. This duality allows for better distinction between noise and meaningful structural patterns.

3.4.2 Continuous Wavelet Transform

For entropy analysis in this study, the Continuous Wavelet Transform (CWT) was employed. CWT is defined as:

$$W(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t) \psi^* \left(\frac{t - b}{a} \right) dt \quad (6)$$

where:

$x(t)$ represents the entropy value at a given game round t ,

$\psi(t)$ is the wavelet function,

a is the scaling factor controlling frequency resolution,

b is the translation parameter shifting the wavelet across different game rounds.

This method reveals how entropy evolves across scales, highlighting both early-stage exploration and endgame convergence (Torrence & Compo, 1998; Addison, 2017).

3.4.3 Implementation

Entropy data across turns was processed using Python's `scipy.signal.cwt` function with Morlet wavelets. Resulting CWT heatmaps illustrate how entropy concentrates or disperses across game phases, offering strategic insights beyond static probability models.

4. Result

4.1 Random Strategy: Result and Analysis

The random strike strategy serves as the baseline model in this project. In this approach, each turn involves selecting a random coordinate from the set of untargeted cells. No historical strike data is used to inform decisions, resulting in a completely non-inferential attack pattern.

4.1.1 Efficiency analysis of Random Strategy

The provided visualization (KDE vs. Normal Distribution Plot) illustrates the distribution of turns required to complete 10,000 simulated games using a completely random strike algorithm on a 14×14 Battleship grid (196 cells) with five ships occupying 22 cells. The analysis focuses on the algorithm's efficiency in terms of decision-making steps (turns per game) and its alignment with theoretical expectations.

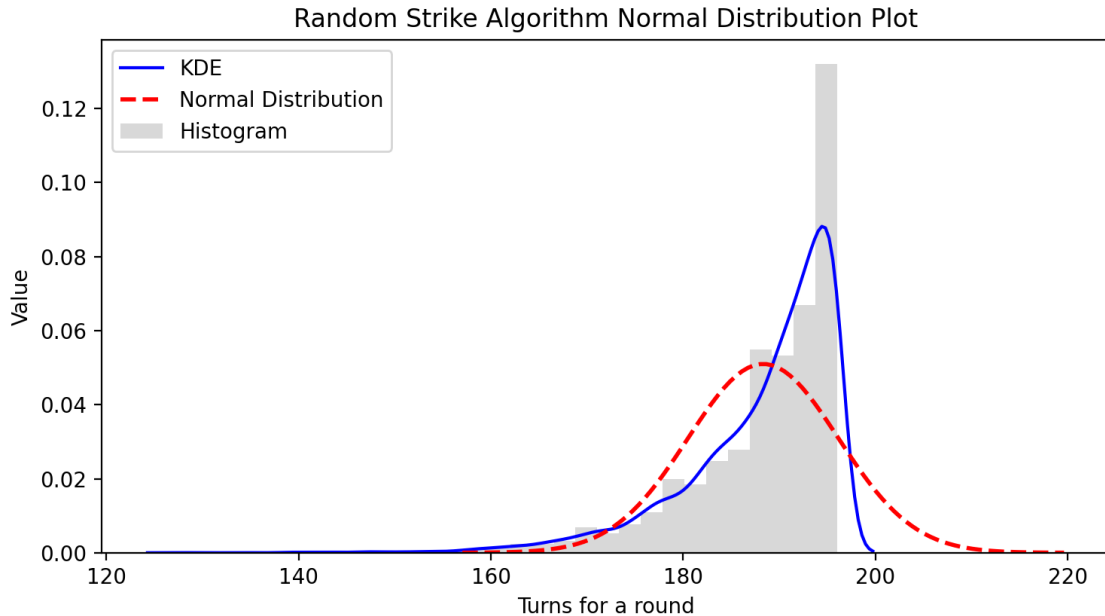


Figure 4 Turn distribution of 10,000 random strategy games

The grey histogram reflects empirical frequencies, the blue curve is a kernel density estimate (KDE), and the red dashed line shows a fitted normal distribution.

Observations:

- The majority of games concluded within **190 to 196 turns**, clustering around a mean near **194 turns**.
- The distribution is slightly **positively skewed**, with a longer tail approaching the upper turn limit.
- The **KDE curve** follows the actual distribution more closely than the fitted normal curve, particularly near the peak and tail ends.

While no game can logically exceed 196 moves under proper implementation, both the **KDE** and **normal distribution fit** are statistical approximations that extend slightly beyond this hard upper bound. These extended values are artifacts of smooth fitting techniques and **do not represent actual data points**.

Interpretation:

- The random strike strategy shows high inefficiency and large variability. With no use of feedback or adaptive logic, the average number of turns required to locate all ships remains close to the maximum possible. The long tail further suggests that poor sampling luck can significantly delay ship discovery.
- This result provides a **baseline benchmark** for comparing more sophisticated strategies and highlights the value of incorporating information-based reasoning into search algorithms.

4.1.2 Entropy Analysis of Random Strike Algorithm

To quantitatively assess the evolution of uncertainty in random gameplay, entropy was computed at each turn using the corresponding probability distribution of untargeted cells. Since the random strategy does not adapt based on prior hits or misses, its probability matrix remains **uniform** at each step, updated only by removing already targeted cells from the set of possible options.

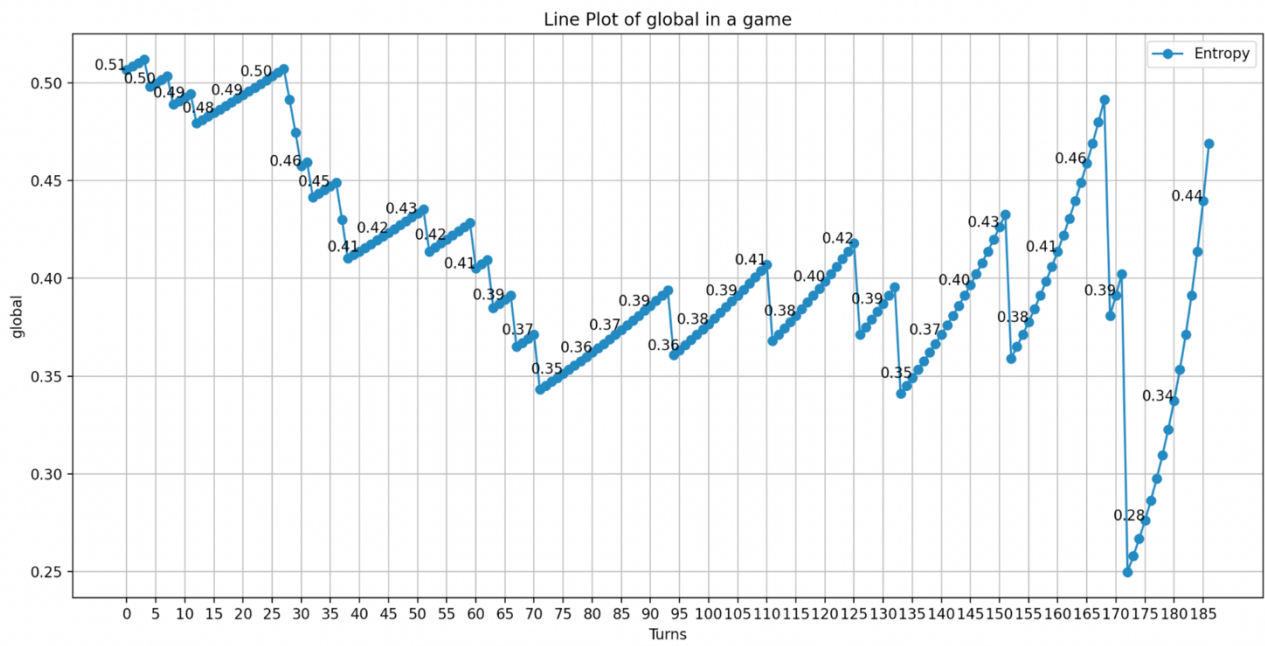


Figure 5 Global entropy trend over a full random game, victory at Turn 186

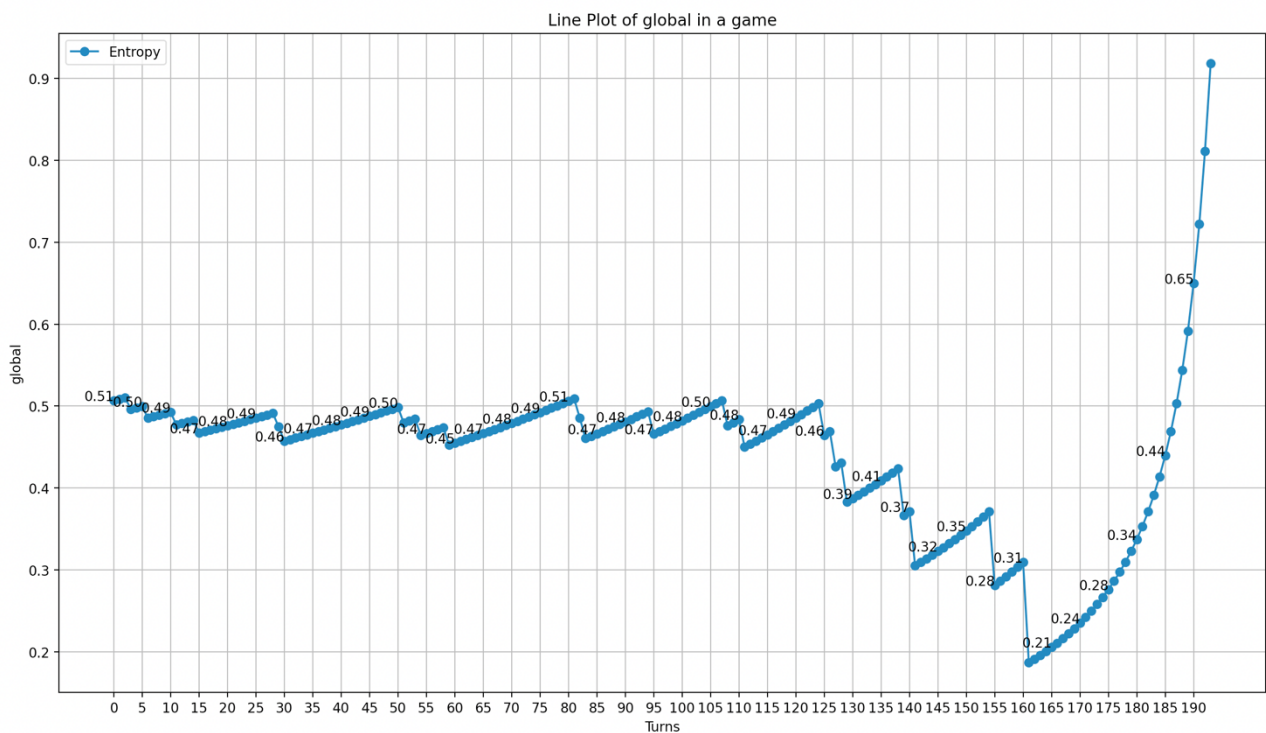


Figure 6 Global entropy trend over a full random game, Victory at Turn 193

Entropy was computed globally at each turn. The values reflect a gradual decline corresponding to the shrinking number of untargeted cells. The endgame region shows slight fluctuations due to the randomness in targeting remaining ship locations.

Observations:

- In both examples, entropy remains within a moderate range (e.g., 0.5 to 0.2), with a slow, staircase-like downward progression.
- Unlike targeted strategies, no sharp dips occur following hits or ship sinkings; entropy continues declining smoothly as more cells are removed from the target pool.
- Some fluctuations are seen in the latter stages of the game—especially in Figure 7, where entropy increases slightly after turn 160. This is likely due to residual unhit ships occupying ambiguous positions near game completion.

Interpretation:

These results reinforce the understanding that entropy under a random strategy reflects only the numerical reduction in unexplored possibilities, not any concentrated information acquisition. It is a passive byproduct of cell elimination, rather than a metric of inference accuracy or strategic success.

The plotted entropy curves provide a visual baseline for comparison against more advanced strategies in subsequent sections. Informed algorithms will exhibit distinct entropy dynamics due to the introduction of non-uniform, feedback-driven probability distributions.

Wavelet-Based Analysis of Entropy Dynamics

To better capture the temporal-frequency characteristics of entropy behavior under the random strategy, a continuous wavelet transform (CWT) was applied to the entropy signals shown in Table 2 and 3. The results are presented below:

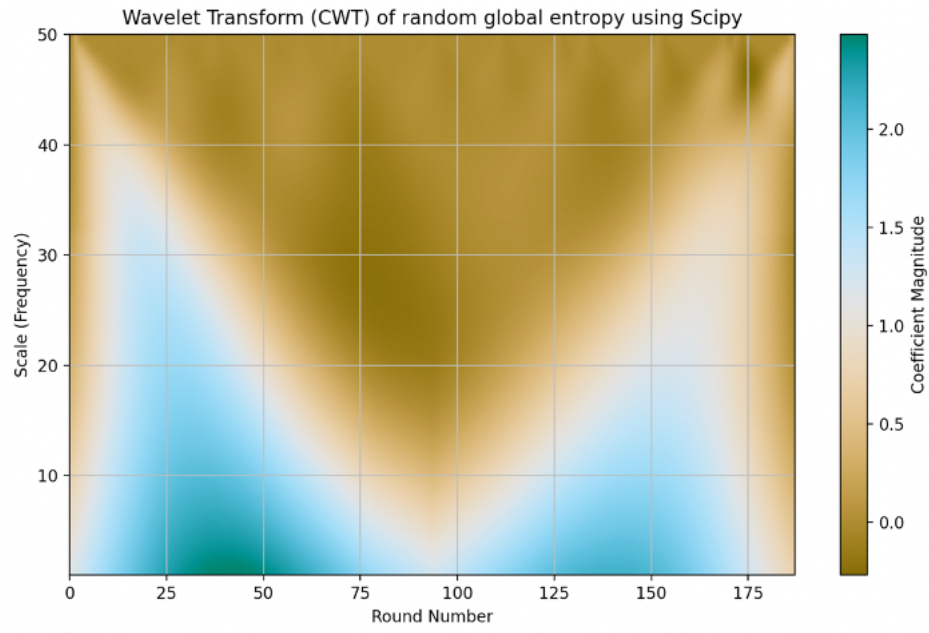


Figure 7 Random CWT of entropy signal from Figure 5

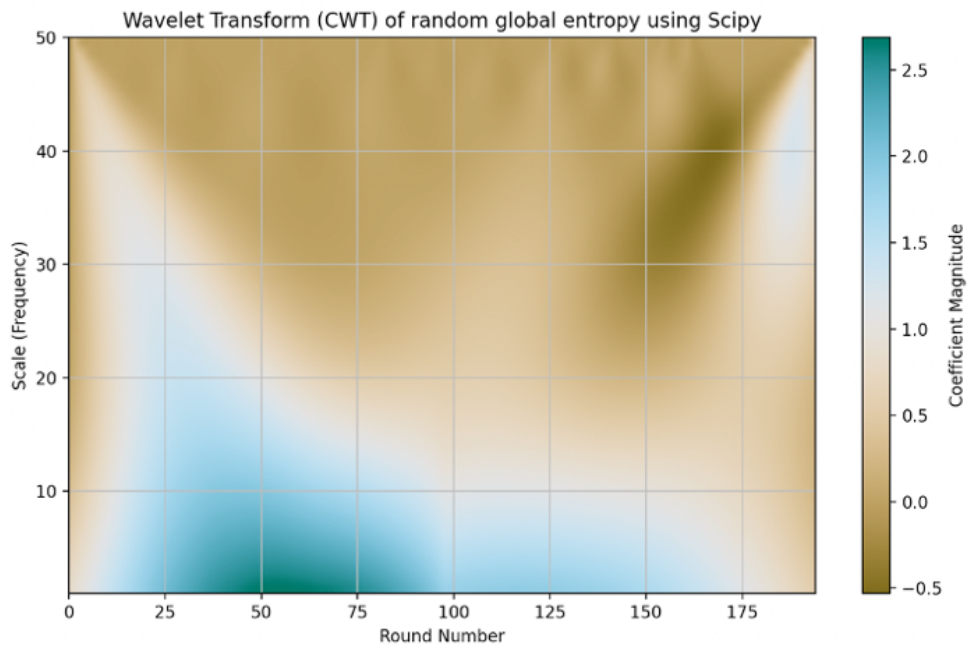


Figure 8 Random CWT of entropy signal from Figure 6

Coefficient magnitudes are color-coded, with blue representing strong fluctuations and grey indicating stability across time and scale.

These heatmaps reveal the following:

- In both cases, **low-frequency structures dominate**, as seen in the red bands located at lower scales early in the game.
- The central portion of the game shows a **relatively stable blue area**, indicating minimal structural change in entropy over midgame turns.
- Toward the end (rightmost region), increased red intensity—especially in Figure 8—suggests **late-stage variability**, likely resulting from final ship discovery.

Interpretation:

Wavelet analysis confirms that while the random strategy introduces some temporal variability in entropy, it lacks consistent high-frequency transitions. The lack of strategic feedback results in smooth, globally modulated entropy trends. These insights further support the conclusion that the random method lacks information-driven adaptability, both in value and structural change.

Further Analysis of Local Wavelet Structures

While both wavelet plots (Figures 8 and 9) share a similar global structure, key differences in local features offer deeper insight into the entropy dynamics under random targeting.

- In Figure 8, the early low-frequency red zone appears as a triangular gradient, with high coefficient concentration from Turns 25–50 **and gradually** expanding across Scales 0–50 until around Turn 80. This shape suggests that early entropy values are declining at a consistent pace, and the red intensity reflects structural stability in the pattern of information removal.
- From Turns 100–175, another **low-frequency band** emerges in Figure 8, forming a **soft red dome-shaped region** extending up to **Scale 40**. While lighter in color, it reflects **continued long-scale regularity** in how entropy declines—indicating that even late in the game, no sudden strategy shift occurred.
- The smoother and more concentrated wavelet structure in **Figure 9**—particularly between Turns 100–175—suggests a more **stable but uneventful** progression. Unlike Figure 8, no

significant high-frequency activity is detected, indicating that the entropy changes gradually with **no major hits or breakthroughs**. This form of stability may in fact reflect **a lack of success in striking valuable targets**.

- Notably, a **sharp red spike** appears at the **end of Figure 9**, between **Turns 190–193**, centered at **Scales 35–50**. This narrow, diagonal “**waterdrop**” or **spike-like signature** suggests a **brief, high-frequency event**—possibly caused by a concentrated burst of decision-making or entropy recalculation just before the final ship was located. Though the random algorithm does not reason logically, such a fluctuation may arise from isolated structural coincidence (e.g., hitting a remaining ship by chance, triggering a late entropy update).

These visual patterns—whether slow decay in entropy values or localized wavelet signatures—do not result from any form of inference, adaptation, or optimization. Instead, they are merely side effects of random target selection. The random strike algorithm **does not make decisions**, but rather eliminates grid cells with equal probability. As such, even visually prominent fluctuations (such as the sharp spike in Figure 9) **should not be interpreted as strategic behavior**, but rather as statistical anomalies resulting from chance.

In this sense, the entropy landscape of the random strategy forms a **control baseline**—a representation of uncertainty **in the complete absence of any decision-making logic or information awareness**. Later strategies will be evaluated in contrast to this purely stochastic behavior.

4.2 Local Possibility Strategy: Result and Analysis

The local possibility strategy introduces a basic probabilistic framework into the game decision-making process. Unlike the purely random approach, this method updates its targeting logic based on confirmed hits by identifying **the most likely surrounding positions to contain the remainder of a ship**. The concept hinges on **local probability inference**, where once a hit is confirmed, nearby tiles are assigned higher priority based on expected ship length and orientation.

This approach simulates a simple human-like reaction: after finding part of a ship, the algorithm assumes the ship continues in adjacent directions and concentrates its subsequent strikes accordingly. While this strategy still lacks full-board analysis or preemptive planning, it **leverages partial feedback** to improve efficiency over purely random exploration.

4.2.1 Efficiency analysis of Probability Base Algorithm

To evaluate performance, 10,000 full game simulations were conducted using the local possibility strategy. The number of turns required to complete each game was recorded and analyzed to construct a statistical profile of efficiency.

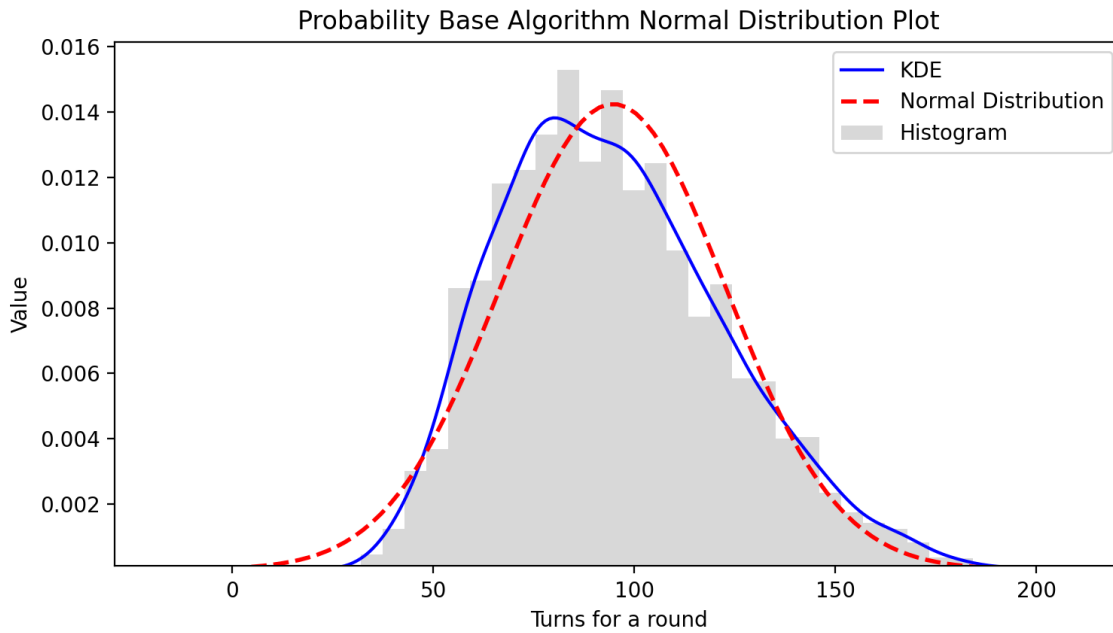


Figure 9 Normal Distribution of Turns for Local Possibility Strategy

Histogram, KDE curve, and Gaussian fit showing the round-based performance over 10,000 simulations

Observations:

- The victory turn count is centered around **95–100 turns**, significantly earlier than the average seen in the random strike algorithm (≈ 190 turns).
- The distribution exhibits a **tight, symmetric bell shape**, indicating **consistent performance and low variance**.
- The **KDE (Kernel Density Estimate)** closely follows the Gaussian curve, further validating the statistical stability of the strategy.

Interpretation:

These results confirm that introducing a **simple local probability model** can substantially reduce the number of required turns to complete a game. By reacting intelligently to hit information, the strategy **reduces redundant strikes** and speeds up the detection of ship orientation and extent.

Compared to the random algorithm, which lacks information exploitation, the local strategy **benefits directly from feedback** and applies it to subsequent decision-making, even though its inference range is still localized and reactive rather than predictive.

4.2.2 Entropy Analysis of Local Possibility Strategy

To investigate how localized probabilistic decision-making affects information gain, this section presents a comprehensive entropy analysis from both **global** and **local** perspectives. Entropy serves as a quantitative indicator of uncertainty; in this context, it reflects how effectively the algorithm reduces ambiguity about the positions of hidden ships.

To enable a robust comparison across varying gameplay dynamics, **three representative cases** are selected based on victory turns: a fast (43 turns), average (97 turns), and slow (145 turns) outcome. These samples reflect different interactions between randomness and local inference, such as:

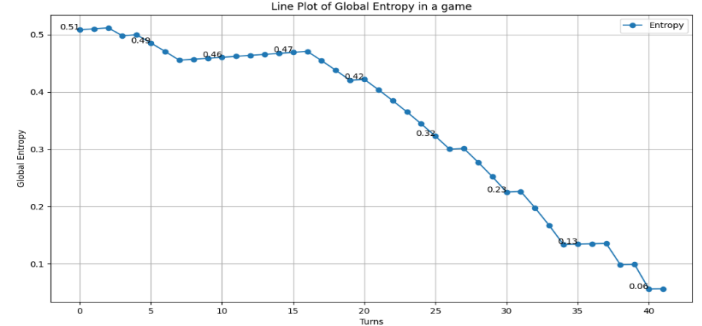
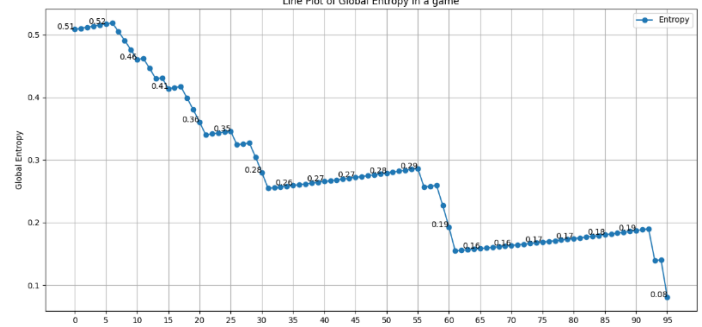
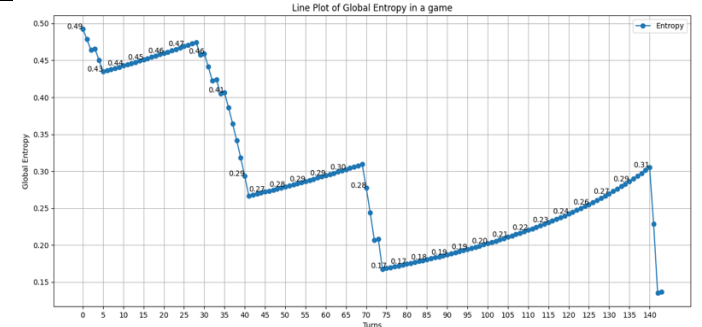
- **Fast case:** early hit, quickly activated local guidance, rapid convergence.
- **Average case:** delayed first hit, moderate targeting accuracy, mixed entropy descent.
- **Slow case:** poor early performance, late access to information, prolonged entropy decay

Each case is analyzed through four perspectives:

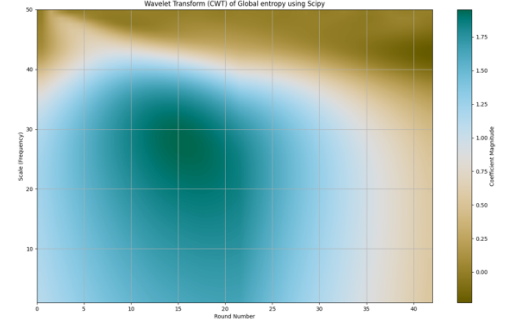
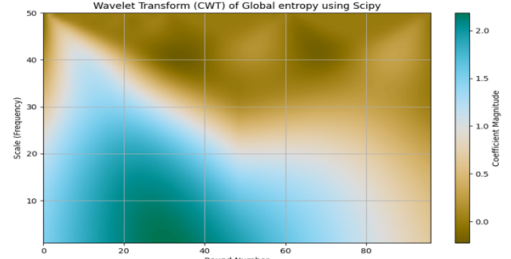
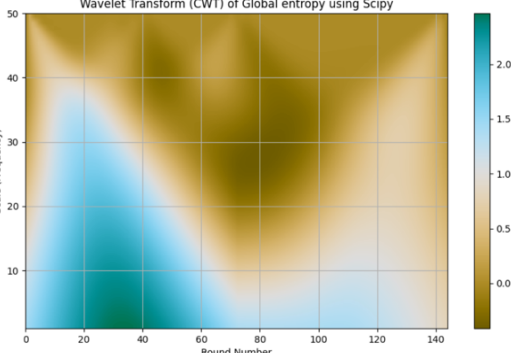
1. **Global Entropy Curve:** Tracking the total information uncertainty over time, capturing the macroscopic efficiency of the algorithm.
2. **Global Entropy CWT (Continuous Wavelet Transform):** Offering a time-frequency decomposition to detect sharp transitions or long-term trends in entropy reduction.
3. **Local Entropy Curve:** Measuring localized information certainty surrounding hits, showing how sharply the algorithm focuses its attacks after feedback.
4. **Local Entropy CWT:** Visualizing how spatial decision patterns evolve in terms of targeting sharpness and responsiveness over time.

All four plots are displayed per case in a structured table format. This integrated comparison provides insights into **how and when entropy collapses**, and how strongly localized reasoning contributes to decision acceleration.

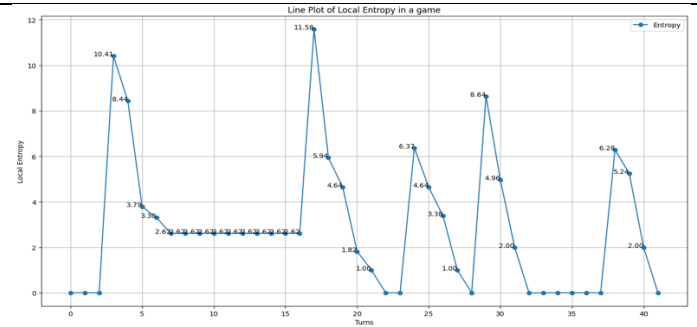
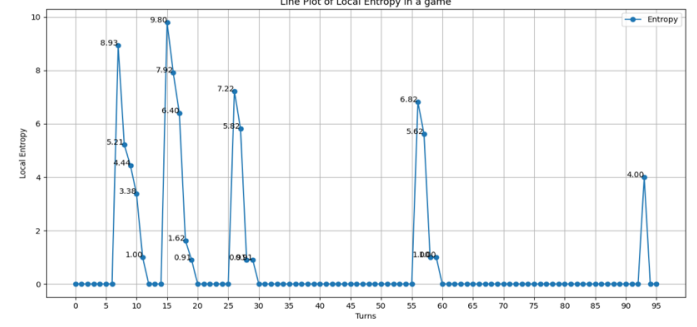
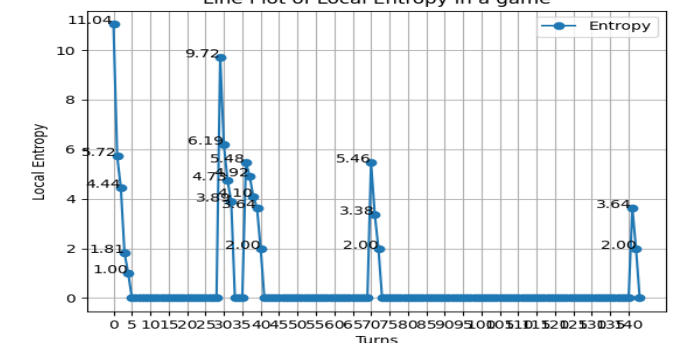
Global Entropy Curve Evolution (Table 4.2.2a)

Turns to win	Description of the Table	Global Entropy vs. Turns Table
43 Turns	Early Random Stage: Before any hits, entropy declines slowly due to random probing. Probability-Guided Stage Post-Hit: Once a hit is confirmed, the algorithm uses local probability to target surrounding tiles, causing a rapid entropy drop. This early convergence demonstrates the strategy's strength when initial contact is established quickly.	 Figure 10 Global Entropy Curve for 43 Turns
97 Turns	Mixed Stage: Entropy declines slowly in early turns but accelerates post-hit. Fluctuations indicate multiple hits triggering local updates. The curve demonstrates how local strategy balances random search with precision targeting once information becomes available.	 Figure 11 Global Entropy Curve for 97 Turns
145 Turns	Delayed Detection: Long delay before any hit results in a flat entropy curve early on. Once a ship is hit, local probability activates and entropy drops quickly. However, the late discovery significantly extends total turns. This reflects the strategy's dependence on early engagement.	 Figure 12 Global Entropy Curve for 145 Turns

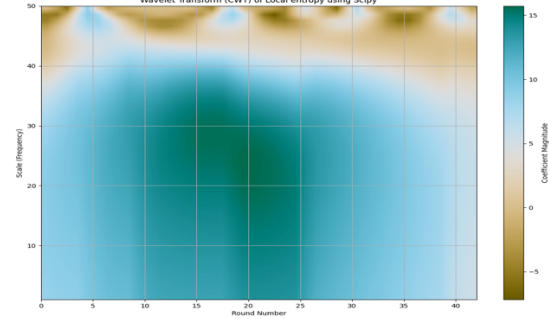
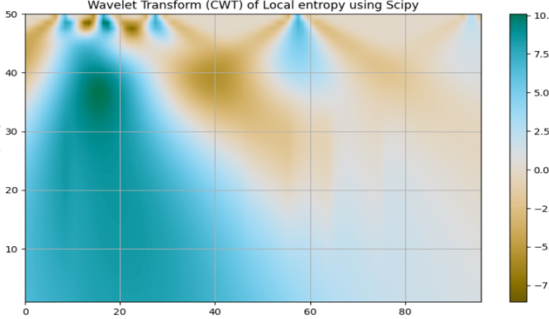
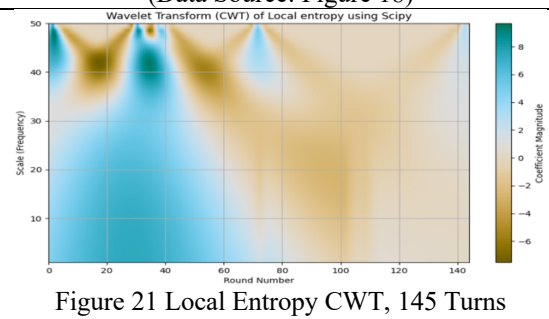
CWT Chart of Global Entropy (Table 4.2.2b)

Turns to win	Description of CWT Chart	Global Entropy CWT Chart (Data Source: Table 4.2.2a)
43 Turns	High-frequency energy concentration is observed as a large red area spanning rounds 0–35, darkest at high-scale ranges (10–20). This reflects the algorithm's transition from exploratory randomness to a rapid post-hit targeting phase. The intense clustering around scale 10–20 suggests a burst of localized probabilistic decision-making, producing immediate and sharp entropy shifts.	 Figure 13 Global Entropy CWT Chart of 43 Winning Turns (Data Source: Figure 11)
97 Turns	A red triangular structure appears in rounds 0–60, with a dark red core emphasizing concentrated low-scale energy. This pattern aligns with the algorithm's strategic cycle: early random attacks (accumulating low-scale energy with long-period entropy fluctuations) transition to targeted strikes post-hit. The triangular core's deep red hue reflects entropy fluctuations focusing energy on low scales during precision guidance, demonstrating coherent strategy evolution from randomness to probabilistic targeting.	 Figure 14 Global Entropy CWT Chart of 97 Winning Turns (Data Source: Figure 12)
145 Turns	Low-Scale Dominated Randomness: A red triangular pattern in rounds 0–60 with intensified low-scale shading indicates prolonged early random attacks, mirroring pure random algorithm characteristics. Entropy fluctuations exhibit low-scale long-period dominance, underscoring inefficient target acquisition during this phase. Attenuated Guidance Strategy: A faint red triangle attempt in rounds 80–140 reflects dispersed energy during late-stage probabilistic targeting. This case highlights the cost of delayed information gain, with entropy structure resembling the random algorithm in early stages.	 Figure 15 Global Entropy CWT Chart of 145 Winning Turns (Data Source: Figure 13)

Local Entropy Graph Evolution (Table 4.2.2c)

Turns to win	Description of Table	Local Entropy vs. Turns Table
43 Turns	The short interval between peaks reflects the immediate transition from random to guided strikes . After each hit, the algorithm quickly switches to local probability calculation, yielding sharp entropy peaks followed by rapid collapses. This pattern confirms the efficiency of the “hit → surround → sink” loop , characteristic of high-performance execution.	 Figure 16 Local Entropy Curve, 43 Turns
97 Turns	Fluctuations occur in multiple phases. In early stages, the hit intervals are moderate; however, in later stages, the interval between spikes grows to 32 turns , indicating prolonged ship detection time . This pattern highlights a mid-performance regime , where partial guidance works, but struggles with slower re-targeting due to limited remaining zones and missed opportunities.	 Figure 17 Local Entropy Curve, 97 Turns
145 Turns	A prolonged interval between early entropy peaks reveals ineffective early exploration . The lack of frequent spikes shows delayed engagement with local inference. Even after hits, low or flattened entropy behavior indicates poor local verification. This suggests that the algorithm fails to capitalize on strike information efficiently, prolonging the decision chain.	 Figure 18 Local Entropy Curve, 145 Turns

CWT Chart of Local Entropy (Table 4.2.2d)

Turns to win	Description of Table	Local Entropy CWT Chart (Data Source: Table 4.2.2c)
43 Turns	Efficient Strategy Transition: The 43-round CWT plot shows dense red energy clustering during early rounds (0–30), especially at mid-to-low scales (10–30). This represents frequent local entropy bursts after each hit, as the algorithm rapidly responds with local probability targeting. The coherent horizontal red zone aligns with sharp local entropy peaks, validating a stable, high-efficiency post-hit routine.	 Figure 19 Local Entropy CWT, 43 Turns (Data Source: Figure 17)
97 Turns	Mid-Stage Delay in Post-Hit Response: The CWT plot for the 97-round game shows moderate red activity in rounds 0–40, concentrated at low-to-mid frequencies (10–30), representing local entropy adjustments during post-hit response. Compared to the 43-round case, red zones are less intense and more temporally spread, suggesting less decisive or delayed post-hit targeting. The absence of strong high-frequency clusters implies that the algorithm experienced slower transitions from hit detection to efficient local strikes, leading to moderate delays in entropy reduction.	 Figure 20 Local Entropy CWT, 97 Turns (Data Source: Figure 18)
145 Turns	Inefficiency in Random Search Stage: A persistent low-frequency red dominance (0–60) appears, indicating ineffective early exploration. Sparse red bursts after round 80, with scattered mid-high frequency responses, highlight a delayed and diffused guidance phase. The flat color distribution reflects disorganized or weak post-hit reaction, affirming that local strategies failed to efficiently guide targeting.	 Figure 21 Local Entropy CWT, 145 Turns (Data Source: Figure 19)

4.3 Advance Global Entropy Analyze

In Section 4.1, a basic form of global entropy was introduced, where the total number of remaining ship segments was divided by the number of unsearched cells. In Section 4.2, local probability + random algorithm is introduced. Even if the probability algorithm is added, winning in a short round still depends on the efficiency of the random algorithm, that is, the luck factor. Obviously, global entropy has not been used very well before and only serves to analyze data. It treats all unsearched cells as equally probable and disregards spatial constraints or ship shape configurations. As a result, it lacks the precision required for more informed decision-making as the game progresses.

This limitation motivates the need for **a more advanced and structured global entropy analysis**. In practice, as hits and misses accumulate, the space of feasible ship placements becomes increasingly restricted — not only in quantity but in spatial pattern. A uniform entropy assumption fails to reflect this evolution and may lead to suboptimal targeting or inefficient use of available information.

Our enhanced analysis addresses this issue by calculating entropy from the full enumeration of valid ship permutations on the current board. Specifically, for each legal configuration of a remaining ship, its footprint is mapped onto the game grid. Every unoccupied cell accumulates a weight based on how frequently it appears across all such legal placements. This generates a probability distribution that accounts for both spatial and logical constraints: forming the basis for a more realistic and high-resolution entropy map.

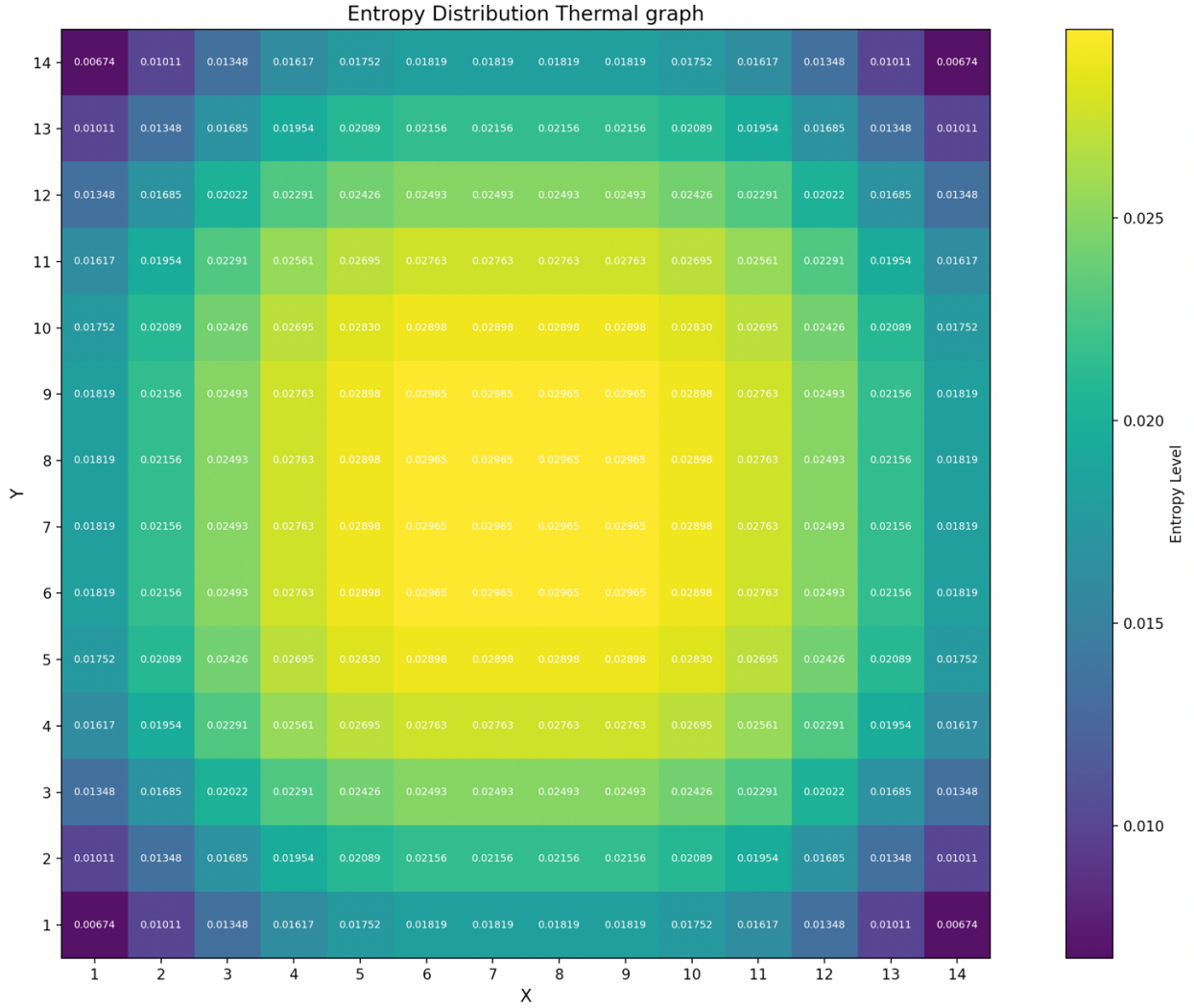


Figure 22 Advanced global entropy heatmap, Associated with Table 1 Data of Each Ship

This refined global entropy estimation offers several key advantages:

1. **Spatial Awareness:** It captures the interaction between known hits, misses, and the board geometry, allowing for the identification of more probable ship locations.
2. **Dynamic Recalibration:** The entropy field is recalculated after each move, adjusting dynamically as new information becomes available.
3. **Visual Insight:** When visualized as a heatmap, this method reveals entropy gradients across the board, highlighting zones of high uncertainty concentration or strategic importance.

By advancing beyond simple ratios, this technique supports smarter decisions grounded in **maximum information gain**. It aligns better with principles from information theory and hidden state estimation, ultimately enhancing the performance of both human players and AI agents.

4.3.1 Heatmap Case Study: Global Entropy Dynamics

In this case study, the entropy-driven strategy autonomously targets the grid cell with the highest global probability at each step, leading to a complete victory in just 48 turns. Starting from an initially uniform entropy distribution, the algorithm dynamically recalibrates its heatmap after every hit or miss, always steering toward the cell whose inclusion in all remaining valid ship permutations is maximal.

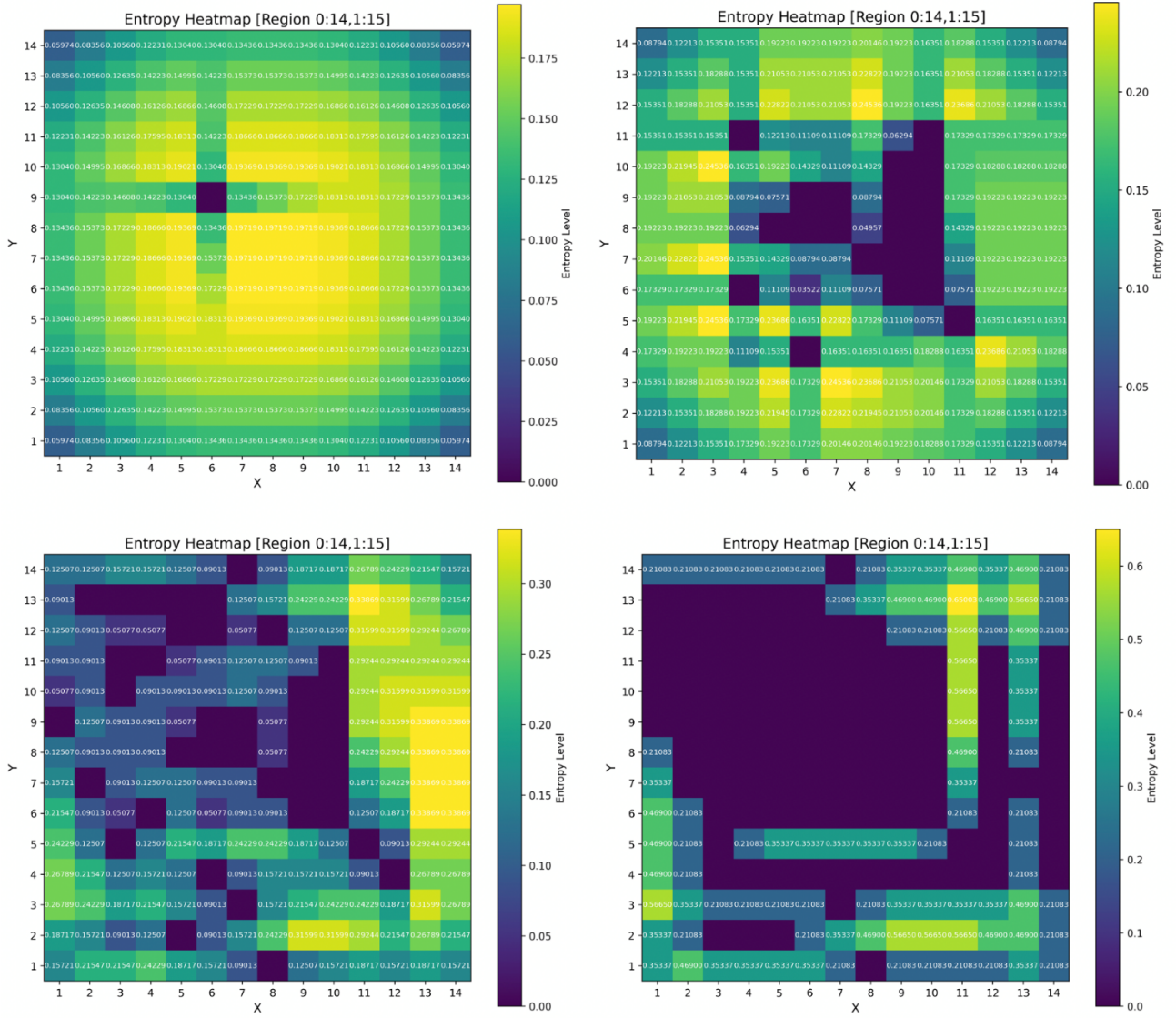


Figure 23 Entropy Heatmap at Turn 1, Turn 18, Turn 34 and Turn 41

The visual sequence of entropy heatmaps in Figure 23 (Turn 1, 18, 34, and 41) illustrates how global entropy dynamically evolves in response to information acquisition during a successful 48-turn game. Each snapshot captures a pivotal stage of transformation in the board's probability distribution, reflecting how information compresses, redirects, and intensifies as uncertainty diminishes.

Turn 1 (Top-Left): The initial entropy distribution is smooth and symmetric, forming a radial gradient centered around the mid-board. This is the result of calculating permutations for all

survival ships across a uniformly unsearched board. As all tiles are equally eligible and unverified, entropy distribution mirrors global ship placement probability, leading to low-to-moderate entropy values in outer tiles and elevated values near the center.

Turn 18 (Top-Right): Several exploratory strikes have occurred, and at least one target has been partially identified. Entropy values become increasingly uneven—certain regions begin to darken (low entropy), corresponding to confirmed misses or confirmed information voids. Meanwhile, concentrated yellow regions (high entropy) begin to cluster near previous hits, indicating intensified uncertainty within narrowed zones. These clusters signal the algorithm's growing confidence in the possible location of ship continuations.

Turn 34 (Bottom-Left): The entropy heatmap reveals a **concentrated high-entropy region** prominently located in the **right-central quadrant** of the board, approximately covering **X = 11–14, Y = 4–11**. This region is marked by **elevated entropy values** exceeding 0.33, shown in **bright yellow**, signifying zones with the highest remaining uncertainty.

What distinguishes this pattern is the **compression effect from the surrounding low-entropy (dark) regions**, especially the **large black (zero-entropy) zones** dominating the **left-center** of the board (X = 3–9, Y = 5–10). These dark regions indicate prior information gains—areas where the algorithm has either struck or eliminated possibilities through deduction—causing the remaining probabilities to be **pushed outward and concentrated** in the right-center zone.

This spatial compression leads to a scenario where the **uncertainty is no longer evenly distributed**, but rather **funneled** into a confined area. As the available possibilities shrink, each remaining valid tile holds more probabilistic weight, reflected by the **heightened entropy values** in this region.

Overall, turn 34 marks a **critical transition phase**, characterized by:

- A **reduction of the global search space**,
- A **tight focus** on unresolved zones with high entropy,

- And a **strategic narrowing** of target predictions as the algorithm converges toward the final ships.

Turn 41(Bottom-Right): The entropy landscape has undergone a dramatic transformation, signaling the nearing end of the game. The heatmap now displays an environment where most of the board—especially the central and left regions—has turned entirely black (zero entropy), indicating that those areas have been fully explored or logically eliminated as potential ship positions.

The remaining entropy is now restricted to a narrow strip along the rightmost columns, particularly $X = 11-13$, $Y = 9-14$, with a few lingering high values in $X = 11$. These bright yellow and green tiles represent the final unresolved candidates for enemy ships.

This configuration exemplifies a highly compressed information structure, where:

- Most of the **entropy mass has collapsed** into a slim vertical corridor,
- The **search space is minimal**, but each remaining tile holds **very high informational weight** (entropy values above 0.6),
- The algorithm has entered a **decisive targeting phase**, where even a single probe could potentially end the game.

In essence, Turn 41 reflects the culmination of cumulative deductions and probability redistribution:

- The high entropy tiles are no longer diffused,
- The **contrast between explored (0 entropy) and unexplored (high entropy)** zones is at its maximum,
- And the board's structure becomes **polarized**, with all strategic focus narrowed to a few final locations.

The advanced global entropy strategy distinguishes itself by integrating **combinatorial ship placement logic** with fine-grained spatial analysis. Unlike random or locally reactive algorithms, this method calculates the **permutation weight of each unsearched tile**, assigning entropy values based on the remaining valid ship configurations across the entire board. As a result, entropy is not merely a by-product of observation but a **direct reflection of quantified spatial possibilities**.

Through sequential heatmap snapshots and comparative grid metrics, it becomes clear that this strategy leads to **systematic information convergence**. Entropy begins with smooth, centralized gradients but quickly contracts toward **spatially compressed zones**, often located near edges or corners. The formation of large 0-entropy regions, driven by continuous elimination, not only reduces the search space but also enhances **decision clarity** in later turns. These patterns are consistent across multiple game instances, indicating a level of **predictable structural behavior** that can inform broader AI decision systems.

In summary, the advanced entropy strategy does not rely on reactive strike patterns or feedback loops. Instead, it builds a **proactive, information-rich environment**, wherein probability is computed, visualized, and acted upon with a high degree of precision. This reinforces the utility of entropy not just as a descriptive tool, but as a **strategic instrument for intelligent control in constrained decision spaces**.

5. Discussion & Conclusion

This project explored the strategic potential of entropy-based decision-making in a simplified Battleship environment by comparing three strike algorithms: **random**, **local probability**, and **advanced global entropy**. Each strategy reflects a progressively deeper level of information usage, illustrating how decision-making efficacy improves with more structured understanding of spatial uncertainty.

The **random strategy** relies entirely on uninformed trial-and-error, treating all unsearched tiles as equally probable. As shown in the entropy heatmaps and distribution curves, this method yields high

overall entropy values throughout much of the game, with limited differentiation across the board. While it serves as a useful baseline, its lack of responsiveness to accumulated data results in inefficient search behavior and wide variability in turn count.

In contrast, the **local probability strategy** incorporates short-term information gained through successful hits. By concentrating efforts around recently discovered ships, it enables localized reductions in entropy and often accelerates the sinking of nearby targets. However, its performance is uneven, as it lacks a global view of ship placement and sometimes overcommits to misleading zones. The entropy progression here is more dynamic, reflecting bursts of rapid information gain, followed by plateaus when incorrect local paths are pursued.

The most sophisticated strategy, **advanced global entropy**, implements a full permutation-based model to calculate the theoretical probability of each tile being occupied by any remaining ship. This allows for entropy to emerge naturally from the structure of possibilities, rather than from reactive events. The result is a tightly controlled narrowing of uncertainty, where 0-entropy regions quickly expand and the remaining probability mass is channeled into highly plausible regions. Notably, this method does not rely on entropy to make decisions—it uses **probability as the core mechanism**, with entropy serving as a meaningful indicator of information clarity at each stage.

A central insight emerges from this comparison: **entropy alone is not the goal**—it is the consequence of how well probability is modeled and updated. Efficient strategies reduce entropy not by design, but by refining the probability field through logical deduction. From this perspective, entropy becomes a valuable **analytical tool**—one that provides both real-time feedback on decision quality and visual insight into the evolution of information gain.

This work ultimately demonstrates that **in constrained search problems**, combining spatial reasoning with layered probability estimation enables faster convergence and more interpretable outcomes. Such entropy-driven approaches hold significant relevance beyond recreational gameplay, offering potential applications in domains such as **autonomous inspection, sensor-driven navigation, and search optimization** in robotics and engineering.

5.1 Further Work

While the current study offers a structured, entropy-driven framework for spatial decision-making, several avenues remain open for future exploration and refinement.

1. Dynamic Permutation Updating

The current permutation system is static, based solely on preset ship sizes and immediate board state. A next step would be to implement a **learning-based adjustment mechanism**, where permutations are updated over time based on observed ship layouts and strike patterns. This could make the model more adaptive, especially under conditions with non-standard or hidden ship configurations.

2. Entropy-Guided Hybrid Strategies

Although the global entropy strategy performs well in isolation, it may benefit from **hybridization with local feedback mechanisms**. For instance, once a ship is partially discovered, switching to a localized entropy focus—rather than continuing globally—could accelerate its elimination while maintaining high global efficiency. Designing such **state-dependent transitions** between global and local entropy usage could create more versatile AI agents.

3. Incorporating Probabilistic Noise and Uncertainty

Real-world applications rarely operate under perfect information. Introducing **probabilistic noise**—such as false positives or missed detections—into the simulation could make the entropy model more robust and reflective of actual engineering problems, such as sonar scanning or structural flaw detection.

4. Scalability to Higher-Dimensional or Non-Uniform Spaces

The grid used in Battleship is regular and uniform. In contrast, industrial inspection, robotic exploration, and autonomous mapping often take place in **irregular or continuous environments**. Extending the entropy modeling framework to **irregular meshes, 3D search spaces**, or time-evolving maps could unlock new applications in fields like **drone path planning, medical imaging, or automated warehouse search**.

5. Real-Time Visualization and Human-AI Collaboration

The visual entropy heatmaps developed in this project have strong interpretability. A future direction is to embed such visuals into **interactive AI systems** that assist human operators—e.g., by highlighting probable regions of concern in maintenance inspection or guiding decision-makers in complex search-and-rescue missions.

By addressing these challenges, the entropy-based approach outlined in this project can evolve from a game-theoretic tool to a **scalable, domain-flexible framework** for intelligent spatial reasoning in real-world engineering and automation contexts.

5. Management

6.1 Original Management

The project was structured across two semesters, with clearly delineated phases including foundational learning, algorithm development, implementation, and final reporting. A Gantt chart was developed and followed to ensure consistent progress and balanced workload throughout the academic year.

Semester 1

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Christmas Holiday
1. Individual Project Proposal	■	■	■										
2. Battleship Battle Algorithm Programming			■	■	■	■							
3. Learning shannon entropy			■	■	■	■	■						
4. Applied Shannon Entropy in Code				■	■	■	■	■					
5. Analysis of different situations and transformations					■	■	■	■	■				
6. Review										■	■	■	■

Semester 2

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Easter Break	Week 11	Week 12
1. Preparing Poster													
2. Preparing Presentation													
3. Developing Local Probability Algorithm													
4. Developing Visualization Distribution													
5. Developing Global Probability Algorithm													
6. Writing Final Report													

Semester 1: Foundational Development and Exploration

The first semester focused on foundational groundwork. Initial efforts in **Weeks 1–3** were devoted to drafting the *Individual Project Proposal*, which defined the project scope and objectives. Simultaneously, **from Week 2 to Week 6**, programming work on the baseline *Battleship Algorithm* began and continued in parallel with self-driven learning on *Shannon Entropy* (**Weeks 3–6**), which served as the theoretical basis for entropy analysis.

By **Week 5**, knowledge was applied in the form of *Entropy Integration into Code*, overlapping with **Weeks 5–8**. This enabled further application of entropy theory into practical coding scenarios. In **Weeks 6–9**, efforts shifted toward the *Analysis of Different Situations and Transformations*, exploring how various strike strategies affect entropy. The semester concluded with a dedicated **review period** (**Weeks 10–12**) to consolidate knowledge and evaluate progress.

Semester 2: Algorithm Enhancement and Finalization

Semester 2 transitioned into the advanced development phase. It began with *Poster* and *Presentation* preparation in **Weeks 1–3**, ensuring early preparation for dissemination tasks.

From **Week 5 onwards**, the focus shifted to more complex tasks:

- *Local Probability Algorithm* development (**Weeks 6–8**)
- *Global Probability Algorithm* development (**Weeks 6–12**)
- *Visualization Tools* (such as entropy heatmaps and wavelet transformations) were introduced and refined between **Weeks 9–11**

Simultaneously, the *Final Report* was progressively written from **Week 6 to Week 12**, allowing for continuous integration of research findings and visual outputs into the final deliverable.

6.2 Reflection and Planning Analysis

The project plan was adhered to with only minor deviations. The two-phase scheduling allowed a gradual build-up of technical capacity, from understanding entropy to developing a game-based entropy-driven decision-making model.

- The **first semester** emphasized learning and experimentation, with effective overlapping of coding and theory acquisition.
- The **second semester** focused on refinement and visual analysis, including integration of algorithms, validation, and graphical outputs.
- The planning successfully incorporated **buffer periods** for review and writing, which reduced deadline pressure.
- Minor schedule shifts occurred due to the complexity of *local/global entropy visualization tools*, which took longer than anticipated; however, early completion of initial algorithm tasks allowed time recovery.

6.3 Risk Assessment and Future Work Recommendations

The development of this project encountered several potential risks that were identified and mitigated during the planning and execution phases. A key risk involved the **increased complexity in algorithmic integration**, particularly when transitioning from simple random strategies to advanced probability-based entropy models. To reduce the impact of this risk, early phases focused on modular development, allowing each component to be tested and validated independently before integration.

Another risk emerged from the **computational load and visual clarity of entropy heatmaps and wavelet transformations**, especially as resolution and game complexity increased. This was addressed through iterative optimization and progressive visualization techniques. However,

visualization bottlenecks occasionally delayed the feedback loop between result generation and interpretation.

There was also a dependency on **accurate algorithm performance under probabilistic uncertainty**, where unexpected edge cases (e.g., fragmented entropy zones) could lead to algorithm misjudgment. This was partially mitigated by incorporating redundancy in entropy calculations and using entropy curves for validation.

For future development, several directions are proposed:

- **Scalability and complexity:** Extending the current model to larger grids or dynamic ship configurations could reveal further insights into algorithm robustness and entropy behavior under variable-scale uncertainty.
- **Interactive decision support:** Introducing real-time user interaction or adaptive AI learning could simulate human-machine collaboration and enrich the strategic depth of the system.
- **Hybrid entropy analysis:** Combining **local and global entropy** with alternative information-theoretic models could further enhance decision accuracy and resolution speed.
- **Benchmarking with human players:** Comparing entropy-guided strategies against actual human gameplay would validate the practical effectiveness of the entropy-based decision system and uncover potential for behavioral modeling.
- **Integration with robotics or automation systems:** Given its foundation in probabilistic spatial search, the framework could be applied to engineering problems such as autonomous inspection, robotic search patterns, or adaptive fault detection in sensor networks.

Overall, while the project remained within scope and achieved the stated objectives, these recommendations present viable paths for extension and cross-disciplinary application.

6. Reference

1. to, C. (2011). *Battleship (Tabletop Game)*. [online] TV Tropes. Available at: <https://tvtropes.org/pmwiki/pmwiki.php/TabletopGame/Battleship> [Accessed 22 Apr. 2025].
2. Rochford, A. (2021). *Playing Battleship with Bayesian Search Theory, Thompson Sampling, and*. [online] Austin Rochford. Available at: <https://austinrochford.com/posts/2021-09-02-battleship-bayes.html>.
3. Audinot, M., Bonnet, F. and Viennot, S., 2014. *Optimal Strategies against a Random Opponent in Battleship*. In: The 19th Game Programming Workshop. [online] Japan Advanced Institute of Science and Technology (JAIST). Available at: <https://www.jaist.ac.jp/~sviennot/publications/battleship-2014.pdf> [Accessed 20 Apr. 2025]
4. Torrence, C. and Compo, G.P. (1998) ‘A Practical Guide to Wavelet Analysis’, *Bulletin of the American Meteorological Society*, 79(1), pp. 61–78.
5. Schwartz, A. (2021) ‘Coding an Intelligent Battleship Agent’. *Medium*. Available at: <https://medium.com/data-science/coding-an-intelligent-battleship-agent-bf0064a4b319> (Accessed: 22 April 2025).
6. Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C. and Oliphant, T.E., 2020. *Array programming with NumPy*. *Nature*, 585(7825), pp.357–362.
7. Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. and van der Walt, S.J., 2020. *SciPy 1.0: fundamental algorithms for scientific computing in Python*. *Nature Methods*, 17(3), pp.261–272.
8. Hunter, J.D., 2007. *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), pp.90–95.

Appendix

Algorithm 1 Pseudocode: Permutation Calculator for Ship Placement

Output: Permutation for one ship

Function: Permutation_Calculator(x_hit, y_hit, length, gB[1])

```
Sort x_hit and y_hit in ascending order
Set blocks ← 0
Set permutation ← 0
// Horizontal Direction
If number of hits < length then
    blocks ← number of hits
Set x_back ← last(x_hit)
Set x_front ← first(x_hit)
If x_hit are not aligned or only one hit then
    for i from 1 to length - len(x_hit) do
        If x_back + i < board limit and gB[y][x_back + i] = 0 then
            blocks ← blocks + 1
        Else
            break
        If x_front - i > 0 and gB[y][x_front - i] = 0 then
            blocks ← blocks + 1
        Else
            break
    End
End
If blocks ≥ length then
    permutationH.append(blocks - length + 1)
End
// Vertical Direction
Reset blocks ← 0
Reset permutation ← 0
Set y_back ← last(y_hit)
Set y_front ← first(y_hit)
If y_hit are not aligned or only one hit then
    for i from 1 to length - len(x_hit) do
        If y_back + i < board limit and gB[y_back + i][x] = 0 then
            blocks ← blocks + 1
        Else
            break
        If y_front - i ≥ 0 and gB[y_front - i][x] = 0 then
            blocks ← blocks + 1
        Else
            break
    End
End
If blocks ≥ length then
    permutationV.append(blocks - length + 1)
End
Return total permutations (horizontal + vertical)
```

End Function

[1]gB: gameBoard

Algorithm 2 Pseudocode: Weight Accumulation Ship Placement

Output: Weight distribution 2D list for one ship

Function:

NumeratorCalculator(x_hit, y_hit, length, w2L[1], pH[2], pV[3], pGB[4])

```
Sort x_hit and y_hit in ascending order
x_far ← first(x_hit)
y_far ← first(y_hit)
state ← 0
// Horizontal check
If x_hit are not aligned or only one hit then
  For i from 1 to (length - len(x_hit)) do
    If pGB[y][x_hit[0] - i] = 0 and (x_hit[0] - i > 0) and
      state = 0 then
      x_far ← x_hit[0] - i
    Else
      state ← 1
    End
  End
End
// Update entropy matrix using horizontal permutations
For j from 0 to pH - 1 do
  For k from 0 to length - 1 do
    If pGB[y_hit[0]][x_far + k] ≠ -1 and ≠ -2 then
      w2L[y_hit[0]][x_far + k] ← w2L[y_hit[0]][x_far + k] + 1
      x_far ← x_far + 1
    End
  End
End
// Reset vertical flags
state ← 0
// Vertical check
If y_hit are not aligned or only one hit then
  For i from 1 to (length - len(x_hit)) do
    If pGB[y_hit[0] - i][x_hit[0]] = 0 and y_hit[0] - i ≥ 0 and
      state = 0 then
      y_far ← y_hit[0] - i
    Else
      state ← 1
    End
  End
End
// Update entropy matrix using vertical permutations
For j from 0 to pV - 1 do
```

```

    For k from 0 to length - 1 do
        If pGB[y_far + k][x_hit[0]] ≠ -1 and ≠ -2 then
            w2L[y_far + k][x_hit[0]] ← w2L[y_far + k][x_hit[0]] + 1
            y_far ← y_far + 1
        End
    End
End
Return w2L
End Function

```

- [1] w2L: weight2DList
- [2] pH: permutation Horizontal
- [3] pV: Permutation Vertical
- [4] pGB: Player Gameboard

Algorithm 3 Global Permutation Estimation via Full-Board Search

Output: Global permutation 2D list

Function: GlobalPermutation(ship_length, playerBoard, permutation2Dlist)

```
permutations ← 0
// Loop over all usable board positions [1:14][0:13]
For x in 1 to 14 do
  For y in 0 to 13 do
    If playerBoard[y][x] = 0 then
      // Horizontal Check
      If x + ship_length - 1 ≤ 14 then
        valid ← True
        For z in 0 to ship_length - 1 do
          If playerBoard[y][x + z] ≠ 0 then
            valid ← False
            Break
          End
        End
        If valid then
          permutations ← permutations + 1
          For z in 0 to ship_length - 1 do
            permutation2Dlist[y][x+z] ← permutation2Dlist[y][x+z] + 1
          End
        End
      End
      // Vertical Check
      If y + ship_length - 1 ≤ 13 then
        valid ← True
        For b in 0 to ship_length - 1 do
          If playerBoard[y + b][x] ≠ 0 then
            valid ← False
            Break
          End
        End
        If valid then
          permutations ← permutations + 1
          For b in 0 to ship_length - 1 do
            permutation2Dlist[y+b][x] ← permutation2Dlist[y+b][x] + 1
          End
        End
      End
    End
  End
End
Return gBprobability, permutations
End Function
```